

Migrating from a legacy engine to MessageFoundry

Early Access · MessageFoundry v0.3.2 · September 2026

Migrate clinical interfaces to MessageFoundry in tested stages, with a rollback path at each cutover. This guide is for analysts and developers using Corepoint, Mirth, Cloverleaf, Rhapsody, or Ensemble.

The process has four phases: **inventory** your interfaces, **map** them to MessageFoundry, **test and compare** results, then **cut over one feed at a time**.

Corepoint, Mirth Connect, Cloverleaf, Rhapsody, and Ensemble are trademarks of their respective owners. MessageFoundry is an independent project and is not affiliated with, sponsored by, or endorsed by any of those vendors. Nothing here grades another vendor's product: the comparisons are **concept translations** meant to help you reuse vocabulary you already know, not claims about what any incumbent can or cannot do.

Why teams migrate

Migration changes how your team stores and reviews interface logic. Check how your current engine exports configuration and which parts need translation.

Keep MessageFoundry configuration in an organization-owned Git repository. Connections, routing, and transforms live there. The engine's database holds operational data. Review changes as diffs and use Git to restore earlier configuration. The same modules run in dev, staging, and production with separate environment values. `messagefoundry init` creates a starter feed, environment files, a continuous integration workflow, and a pinned engine version. Keep the engine as an unchanged dependency. Releases include signatures and a software bill of materials.

Routing and transform logic runs as Python. Wizards, snippets, a cookbook, and the Steps view can help write it. Connection transport settings can be edited in a form. The resulting Handler remains a `.py` file that you can review and test.

Before you start: the mental-model shift

MessageFoundry has no channel object.

A bundled channel becomes a set of components linked by name. These are the four building blocks:

Building block	What it is
Connection	An endpoint that <i>receives</i> (inbound) or <i>sends</i> (outbound) messages. Seventeen connector types are available. They include MLLP, raw TCP, X12, local/UNC files, SFTP, FTP/FTPS, HTTP, REST, SOAP, and database. Other types include FHIR, DICOM, DICOMweb, SMTP, Direct, a timer, and two internal connectors. Every message in or out is counted and logged.
Router	A small pure function bound to one inbound. It sees every received message and returns the name(s) of the Handler(s) to forward to. It may filter by returning nothing.
Handler	A pure function that receives a Router's message, then filters and transforms it. It returns Send objects for outbound Connections.
Message store	The durable queue and persistence layer — SQLite by default, or PostgreSQL / SQL Server for production.

The loader resolves component names when configuration loads. An inbound names its Router. The Router returns Handler names. Each Handler **Send**s to a named outbound. Components do not hold direct references to one another.

Split each existing channel into an inbound Connection, a Router, Handlers, and outbound Connections. Define shared destinations and transforms once, then refer to them by name.

Phase 1 — Inventory your existing interfaces

Before touching MessageFoundry, build a complete picture of what you run today. For each interface, capture:

- **Source** — the transport and direction (MLLP/TCP listener, file drop, database poll, HTTP/SOAP endpoint), the listen port or path, and the upstream partner/system.
- **Destinations** — every downstream the channel delivers to, with transport, host/port, and any per-destination filtering.
- **Message types** — the HL7 v2 trigger events (ADT, ORM, ORU, SIU, DFT, MDM, VXU, ...), or X12 / FHIR / other payloads.
- **Routing logic** — the rules that decide which messages go where (often a filter on **MSH-9**, a facility, or a sending application).
- **Transforms** — field mappings, value-set translations (code sets), segment add/remove, and any enrichment from a lookup.
- **Acknowledgement behavior** — original vs. Enhanced ACK, and whether the partner expects application-level NAK semantics.
- **Security** — TLS/mTLS on the wire, credentials, and any IP allowlisting.
- Record peak-hour throughput, rather than the daily average. Identify steady or intermittent feeds. Record whether delivery requires strict ordering.

Record each channel's distinct destinations and transforms. Identify pieces shared across channels so you can define them once. Also flag strictly ordered feeds with high message rates. Ordering requires serial delivery. Consider splitting these feeds at the source, as described under *Deployment guidance*.

Phase 2 — Map the concepts

Use this table to map familiar terms to MessageFoundry components. Terms vary by product and version, so confirm the mapping for your existing engine.

Incumbent concept	MessageFoundry equivalent
Channel (Mirth) / Interface	A path from inbound Connection to Router, Handlers, and outbound Connections. Define these components separately and link them by name.
Source connector (Mirth)	An inbound Connection (<code>inbound(...)</code>).
Destination connector (Mirth)	An outbound Connection (<code>outbound(...)</code>).
Source filter / channel filter	A Router — returns the Handler name(s) to forward to, or nothing to drop the message (recorded as <code>UNROUTED</code> , never silently lost).
Transformer / destination transformer	A Handler — filters, transforms, and <code>Send</code> s.
Destination-set routing / "send to these destinations"	A Router that fans out (<code>return ["to_a", "to_b"]</code>) or a Handler that fans out (multiple <code>Send</code> s).
"E Process" (Corepoint)	A Router — often grouped in a <code>routers_<area>.py</code> file, each listing its handler(s).
"E Child" (Corepoint)	A Handler — a shared transform defined once in <code>handlers_<partner>.py</code> and named by multiple routers.
A channel that is defined but switched off	Set <code>deployed=false</code> to retain configuration without building a connector or binding a listener. The engine does not resolve its environment values. This permits configuration before partner credentials exist. With <code>auto_start=false</code> , the engine builds the connector but waits for an operator to start it.
Configuration held inside the engine	Keep configuration in Git. The database holds messages, operational state, users, and audit records. It does not hold routing or transform code.
Embedded JavaScript / proprietary scripting	Ordinary Python you own, plus a built-in structured HL7 transform model (read/set by field path, iterate repetitions, add/remove segments, MSH-aware re-encode with correct escaping).

A worked example

Suppose your incumbent has a channel "ACME ADT → EHR": it listens for ADT over MLLP, drops anything that is not an ADT, and forwards to your EHR. In MessageFoundry that is one small module:

```

from messagefoundry import MLLP, Send, env, handler, inbound, outbound, router

inbound("IB_ACME_ADT", MLLP(port=2600), router="acme_adt_router")
outbound("OB_EHR_ADT", MLLP(host=env("ehr_host"), port=env("ehr_port", cast=int)))

@router("acme_adt_router")
def route(msg):
    return ["acme_adt_handler"] if msg["MSH-9.1"] == "ADT" else [] # non-ADT → UNROUTED

@handler("acme_adt_handler")
def handle(msg):
    # filter / transform here
    return Send("OB_EHR_ADT", msg)

```

`env(...)` resolves the downstream host and port from `environments/<env>.toml` at load. The same module runs in dev, staging, and prod. A missing referenced value makes the engine reject the graph. Use `[TYPE]_[PARTNER]_[MESSAGE]` names, such as `IB_ACME_ADT` for inbound and `OB_EHR_ADT` for outbound connections.

MLLP listeners take **only a port**. Operators choose the bind interface per environment through service settings. Loopback is the default.

Connections can be data, not just code

Store transport type, settings, router binding, and delivery options in `connections.toml` if you prefer data-based editing. Edit it by hand, through the command line, or in VS Code. Routing and transforms stay in Python. Both forms produce the same registry, validation, and outbound access checks.

Eleven transports are reachable from TOML today — MLLP, TCP, HTTP, file, timer, REST, database (write and poll), SOAP, SFTP and FTP. The rest (X12, FHIR, DICOM, DICOMweb, email, Direct, and the two internal inbounds) are code-first only for now and are declared in a `.py` module. Secrets are never written inline. They are referenced from the environment.

Mapping the transport for each connector

MessageFoundry ships seventeen connector types. Check the ones your feeds actually depend on against this list before you plan a wave:

Incumbent connector	MessageFoundry connector
MLLP / LLP Listener and Sender	<code>MLLP(...)</code> — inbound listener and outbound sender, with optional TLS and mutual TLS.
TCP Listener / Sender (custom framing)	<code>Tcp(...)</code> — raw TCP with configurable delimiter framing, for X12 or other non-HL7 feeds carried opaquely.
X12 over TCP	<code>x12(...)</code> — frames by the interchange itself (ISA...IEA), with optional synchronous request/response (e.g. 270 → 271 real-time eligibility) and TA1 classification on a capturing outbound.

Incumbent connector	MessageFoundry connector
File Reader / Writer	<code>File(...)</code> reads or writes local or UNC directories. It supports size caps, malformed-file quarantine, atomic writes, and process-in-place for read-only shares. An alternate Windows share credential is optional.
FTP / FTPS / SFTP Reader and Writer	<code>Ftp(...)</code> (stdlib. <code>tls=True</code> is FTPS with a verifying certificate check) and <code>Sftp(...)</code> (the <code>[sftp]</code> extra, host-key verification on by default) — each is both a source and a destination.
HTTP Listener	<code>Http(...)</code> accepts HTTP/1.1 POST requests with JSON, XML, SOAP-envelope, or FHIR bodies. It returns <code>202 Accepted</code> after it stores the body.
HTTP Sender	<code>Rest(...)</code> — outbound HTTP(S) client.
Web Service Sender	<code>Soap(...)</code> — outbound SOAP, including an opt-in WS-* mode with mutual TLS, WS-Security, and WS-Addressing.
Database Reader / Writer	<code>DatabasePoll(...)</code> (inbound poll) and <code>Database(...)</code> (outbound write) — always-parameterized SQL over ODBC. SQL Server is the production preset. A generic dialect reaches any database with an OS-installed ODBC driver. No JDBC — there is no JVM.
FHIR client	<code>FHIR(...)</code> — outbound FHIR REST client (create/update/transaction/batch) with conditional create/update knobs for idempotency, and SMART Backend Services client authentication.
DICOM Listener / Sender	<code>DICOM(...)</code> — inbound C-STORE SCP and outbound C-STORE SCU / C-ECHO (the <code>[dicom]</code> extra) — plus <code>DICOMweb(...)</code> for STOW-RS. Headers and Structured Reports only, no pixel data .
SMTP Sender	<code>Email(...)</code> / <code>SMTP(...)</code> , and <code>Direct(...)</code> for Direct-Project S/MIME over SMTP.
Scheduled / clock-driven channel	<code>Timer(...)</code> — interval or cron. Each tick emits an operator-configured body.
Channel Reader / channel-to-channel	The graph itself, plus two first-class internal inbound: <code>Loopback()</code> (a captured synchronous reply re-ingressed with its own Router) and <code>PassThrough()</code> (a 1:N internal hop a Handler <code>Send</code> s into).

These connections are not available:

- S3 or cloud blob storage.
- POP3/IMAP email reading. SMTP sending is available.
- JMS, IBM MQ/MSMQ, and Kafka.
- An inbound FHIR server facade.
- Synchronous SOAP-envelope replies from the inbound listener.
- Request authentication on the HTTP listener's own socket.

If a feed requires one, schedule it for a later stage or use an available transport through another component. Place an exposed HTTP listener behind a proxy that authenticates requests. Serial RS-232 and ASTM lab-instrument connections are excluded by design. They are not pending features.

Handlers can read and write JSON, CSV/delimited, and fixed-width data with Python's standard library. HL7 v2, X12, FHIR, generic XML, and DICOM headers/SR have modeled parsing and validation. C-CDA, NCPDP, and HL7 v3 currently pass as opaque bytes. They support pass-through routing but lack modeled field-level transforms.

Two reliability facts to design around

Two engine behaviors shape how you write transforms and how downstreams should behave:

- **At-least-once delivery.** The engine stores messages before acknowledgment. If a peer processes a send but its acknowledgment is lost, the engine retries. Retries use the stored payload and the **same MSH-10 control ID**. Receivers must tolerate duplicate delivery through upserts, idempotency keys, or message-ID checks. FHIR conditional create/update and X12 TA1 handling support this behavior.
- **Routers and transforms must be pure.** They must return the same output on a safe rerun, without external side effects. Two read-only exceptions exist inside live Handlers: database lookups and FHIR reads/searches. Use these for enrichment or gating. Move other external work into destinations.

Phase 3 — Test and validate in parallel

Validate the configuration and compare message outcomes before cutting over a feed.

Validate the configuration itself

Run `messagefoundry check` in commit hooks and continuous integration. It catches missing routers or handlers, duplicate names, and port conflicts. With fixtures, it also dry-runs sample messages. It builds connectors under the instance's security settings to catch startup refusals before deployment. Lint and type checks are advisory.

Use `messagefoundry impact` to find references to a router, handler, or connection. It can plan and apply a rename across the object and its references.

Dry-run transforms with before/after diffs

The VS Code **Test Bench** runs `.hl7` files through Routers and Handlers without delivery. Its before-and-after diffs let analysts compare each transform with the existing engine. Use the synthetic message generator to avoid real PHI during testing.

Probe connectivity without sending a message

`POST /connections/{name}/test` creates a new connector for a **reachability test**. It does not use the live connector or send a clinical message. It follows the outbound allowlist and records an audit event.

The test depends on transport:

- MLLP/TCP/X12: socket connection.
- Database: `SELECT 1`.
- REST/SOAP: HTTP `HEAD`.
- FHIR: metadata read.
- DICOM: C-ECHO.

- Email: connection and NOOP.
- SFTP/FTP: connection.
- File: directory write-access check.

Listeners, timers, and internal inbounds report "nothing to probe". HTTP **401** and **403** responses count as failures. Use these results to check firewall rules, credentials, and TLS before cutover.

Run a true parallel comparison with the tee relay

Compare MessageFoundry with the existing engine before cutover, using representative **test or synthetic data**. The standalone **tee relay** forwards that traffic to both engines:

- Repoint the upstream source (for example an EHR's outbound) at the relay. The relay acknowledges receipt, then forwards unchanged bytes to both engines: the existing production engine and the shadow MessageFoundry instance.
- Optionally, configure the existing engine to copy its outbound messages to a second relay listener. The shadow MessageFoundry instance can then compare that output.

Set the shadow instance's outbound connections to **simulate mode**. The engine runs routing and transforms, records intended output, and finalizes messages while suppressing delivery. Compare that output with the existing engine before moving a feed.

Observe these relay limits:

- **It is a validation tool for test and synthetic data only.** The relay is not hardened for production PHI. It prints a warning at each startup. Use only a trusted test segment. Use it to gain confidence, not as a permanent production component.
- The relay acknowledges receipt to the upstream sender. Application-level NAKs from the existing engine do not return to that sender. The relay logs every one instead, which is exactly the record you want during a parity run.
- It is a **fail-closed relay, not durable store-and-forward**. If the production path fails, the relay stops accepting new connections. The upstream sender then detects the outage and queues messages. You restart it once the production path is healthy.
- The shadow leg is decoupled, so a slow or down shadow MessageFoundry never back-pressures the production path.
- It records metadata per forwarding leg: outcome, ACK code, control ID, message type, size, and reason. JSON export contains this metadata without message bodies.

Phase 4 — Stage the cutover, keep rollback one step away

Migrate **one feed at a time** using this sequence:

1. **Author and review** the feed's module(s) in your config repo as an ordinary pull request, gated by `messagefoundry check`. If the partner is not active, commit the feed with `deployed=false`. It remains in the graph and documentation. It binds no listener and requires no partner credentials.

2. **Validate** it with the Test Bench (transform parity) and the connectivity probe (every downstream reachable).
3. **Shadow it** through the tee relay in simulate mode and compare output to the incumbent until you are satisfied.
4. **Cut over** by repointing the upstream source at the MessageFoundry inbound and enabling real (non-simulated) egress for that feed's outbounds. Keep the incumbent's channel configured but idle.
5. Monitor the feed during a soak period through the web console at `/ui`. Inspect results, delivery and audit records, alerts, and dead letters. Use replay when needed. Every message lands in exactly one of seven dispositions (received, routed, unrouted, filtered, processed, error, or not-deployed), so "where did that message go?" always has an answer.
6. **Roll back** if anything looks wrong: repoint the source back at the incumbent (or, during the shadow phase, simply stop the relay). The existing channel remains available for rollback. The MessageFoundry configuration change remains a reviewed diff.

Configuration and operational data have separate recovery paths. Promoting configuration leaves stored messages in place. Restoring the store leaves routing unchanged. For disaster recovery, redeploy configuration from Git and restore data from a database backup.

Sequencing the waves

A pragmatic order:

1. Start with a low-risk feed, such as one-to-one ADT pass-through or a file feed. Test the full process (author → check → test → shadow → cut over → monitor).
2. **Then consolidate the duplicated ones.** If feeds share a transform or destination, define that Handler or outbound once. Reference it from each feed.
3. Migrate complex or roadmap-dependent feeds last. These include synchronous request/response and WS-* mutual-TLS submissions. They also include feeds that need unsupported modeled formats or connectors.

Deployment guidance to plan for

Decide these deployment settings before cutover:

- **Bind interfaces deliberately.** Inbound MLLP/TCP listeners bind to a configured interface (loopback by default), overridable per connection. Exposing a listener off loopback is a per-environment operator decision, and the engine enforces it: a non-loopback plaintext listener is refused at start unless you explicitly override. A per-connection source IP allowlist can further restrict which peers may connect. Treat this as deployment configuration, not a limitation.
- **Egress is allowlisted, fail-closed.** An allowlist controls outbound hosts. Add every downstream destination before migration. The engine rejects unlisted destinations.
- **Pick your store for the target scale.** SQLite (zero-setup, single file) is the bundled default and suits pilots and single-node deployments. PostgreSQL or SQL Server is the production choice and is required for high availability.

- **Measure capacity with your partners.** The published baseline records measured results for a named reference configuration. It includes a method that you can repeat. Those measurements do not guarantee performance on your hardware. A target is not a measurement. For strict ordering, each send waits for the partner acknowledgment. A 50 ms response time therefore limits that connection's rate. The reference laboratory measured about 60 messages/second end to end on an ordered connection. Intake was several times faster. Divide busy feeds at the source and size for the peak hour. Engine sharding divides connections across processes with one unified store.
- **High availability is active-passive.** Identical engine processes share one server database. One leader processes messages, and warm standbys take over after failure. Use a floating VIP or load balancer with a TCP-connect check on each listener port. Failover takes time. Clean switchover is prompt. Crash failover depends on the leadership lease, about 30 seconds by default. You can tune that value. Downstream systems must tolerate duplicate delivery.

Security posture during and after migration

Review these controls before moving patient data:

- **Authentication and RBAC** at a single API choke point, with deny-by-default per-route permissions and a tamper-evident audit log. Every PHI access (raw view, replay) is recorded against the acting user.
- **Multi-factor authentication** for local accounts is on by default and covers every local account, with recovery codes. Browser passkeys are available as an alternative second factor. Active Directory users' MFA stays delegated to your own identity provider.
- **Transport security is built and opt-in per surface:** TLS (and optional mutual TLS) on the engine's own API and WebSocket, and MLLP-over-TLS per connection, with outbound peer verification on by default. Turn it on for anything that leaves the box.
- Interface authentication supports mutual TLS and OAuth 2.0 client credentials for HTTP-based outbounds. FHIR endpoints can use SMART Backend Services. Note the gap named earlier: The inbound HTTP listener has no request authentication on its socket. Put exposed listeners behind an authentication proxy.
- **Encryption at rest** — message bodies are encrypted in the store, with key rotation, on all three backends.
- **On-premises by default** — the engine API binds to loopback and requires authentication. No PHI leaves the local environment without explicit, reviewed configuration.

MessageFoundry has a documented **OWASP ASVS 5.0 Level 3** assessment. Each control is implemented or has a written remaining risk. It is a **point-in-time, AI-assisted self-assessment, not a certification, audit, or independent review**. No pass/fail count is published while scoring is reconciled. **No third-party assessment, penetration test, or dynamic testing** has been completed. The dated risk acceptance becomes void on production or off-loopback exposure. Adopters should request independent review. The assessment is private and available to evaluators under NDA. The public security-documentation policy explains what is published and withheld.

On HIPAA, the accurate statement is narrow: MessageFoundry provides the technical safeguards the Security Rule expects of software in this role — access control, audit controls, integrity, and encryption in transit and at rest.

Compliance is a property of a covered entity's whole deployment and program, assessed by that entity and

its counsel. No engine, this one included, confers it.

A short pre-flight checklist

- [] Every incumbent channel inventoried: source, destinations, message types, routing, transforms, ACK behavior, security, peak-hour volume, ordering requirement.
- [] Each channel decomposed into inbound Connection(s), Router(s), Handler(s), and outbound Connection(s).
- [] Check every required transport and format against the connector list. Schedule unsupported dependencies for a later stage.
- [] Shared transforms and destinations identified and defined once, referenced by name.
- [] Config repo scaffolded (`messagefoundry init`). `environments/<env>.toml` set per environment. Secrets supplied from the environment, never committed.
- [] `messagefoundry check` green. Transforms confirmed with the Test Bench against synthetic messages.
- [] Connectivity probed to every downstream. Egress allowlist populated. TLS configured for any off-loopback listener.
- [] Parallel run completed through the tee relay in simulate mode, output compared to the incumbent (test data only).
- [] Per-feed cutover plan with the incumbent channel left idle for fast rollback. Not-yet-live partners committed as `deployed=false` .
- [] Monitoring, alerts, and dead-letter replay verified in the web console. Soak period defined.

Further reading

- MessageFoundry connectors and settings:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/CONNECTIONS.md>
- The mental model — the four building blocks and how a message flows:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/MENTAL-MODEL.md>
- Understanding throughput, and how to size against your own systems:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/THROUGHPUT.md>
- The tee relay for parallel-run validation:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/TEE-RELAY.md>
- Security model and PHI handling:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/SECURITY.md>
- What security documentation is public, what is withheld, and how to ask:
<https://github.com/MEFORORG/MessageFoundry/blob/main/docs/SECURITY-DOCS-POLICY.md>

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.